MIND MIRROR

Deployment
&
User
MANUAL

Team Mind Mirror

Table of Contents

1. Introduction

Welcome to the Mind Mirror Deployment & User Manual. Mind Mirror is an AI-assisted journaling and emotional wellbeing platform that helps users reflect on their daily experiences, track their mood and sentiment over time, and (optionally) share their journal entries and mood trends with their therapist for clinical context.

This full-stack application consists of a Python/FastAPI backend (REST API), a React (Vite) frontend (user interface), and uses Hugging Face transformer models for on-server sentiment and emotion analysis. Optional Groq LLM integration provides personalized written suggestions, and MongoDB Atlas is used as the primary database.

This manual is intended for developers and system administrators who want to set up Mind Mirror for local development and deploy the application to production. It provides step-by-step instructions for configuring the development environment, setting up all necessary services (database, AI models, optional LLM provider), provisioning therapist accounts securely, and deploying both the frontend and backend to the cloud. By following this guide, you will be able to run the application locally for testing and successfully deploy the full-stack application to a live environment.

Scope of this Manual:

- System Requirements — hardware/software requirements for development and production environments.
- Development Setup — setting up the backend and frontend on a local machine for development and testing.
- Configuration — setting up environment variables, database connections, and third-party service credentials (.env for local use; cloud platform environment variables for deployment).
- Therapist Provisioning — securely creating therapist accounts using an admin-only endpoint so that arbitrary users cannot self-assign clinical privileges.
- Deployment Procedure — step-by-step instructions to deploy the frontend (Vercel) and backend (Render) to the cloud.
- User Guide — how patients and therapists use the deployed application end-to-end.
- Verification & Testing — how to run the application and verify that all components (frontend, backend, database, NLP models) work together.
- Troubleshooting — common issues during setup or deployment and how to resolve them.

Whether you are running the application locally or deploying it to the cloud, this guide will ensure a smooth setup and deployment process for Mind Mirror.

2. System Requirements

To successfully develop and deploy Mind Mirror, your system and tools should meet the following minimum requirements. This includes hardware capabilities for running the app locally (note that the NLP models add a moderate memory overhead), software installations for development, and accounts or services needed for production deployment.

2.1 Hardware Requirements

The hardware requirements for running Mind Mirror are slightly higher than a typical web app because the backend loads two Hugging Face transformer models (sentiment and emotion classifiers) into memory at startup.

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz (Intel i5/i7 or Apple M1+)
Memory (RAM)	8 GB	16 GB or more
Storage	10 GB free	20 GB free (model caches, logs, dependencies)
Internet	Broadband connection	Stable high-speed (for first model download and Atlas)

Note: On first launch the backend downloads the Hugging Face model weights (~500 MB total) and caches them under ~/.cache/huggingface. Subsequent launches load from disk and do not require internet for the model itself. A stable internet connection is still required for MongoDB Atlas and (optionally) Groq API calls.

2.2 Software Requirements

Ensure the following software is installed on your development machine:

- **Operating System:** Windows 10/11, macOS 12+, or modern Linux. The project is OS-agnostic.
- **Python 3.11+:** Required for the backend. Verify with `python --version`.
- **Node.js 18+ and npm:** Required for the React (Vite) frontend. Verify with `node -v` and `npm -v`.
- **Git:** For version control. Alternatively, download the project as a ZIP from GitHub.
- **Code Editor/IDE:** Visual Studio Code is recommended. Other options: PyCharm, WebStorm, Cursor.
- **Web Browser:** Chrome, Firefox, or Edge (latest versions) for development and debugging.

2.3 Development Environment

In a development environment (your local machine), you will use the above tools to run the application locally:

- **Python virtual environment:** Strongly recommended (venv or conda) to isolate backend dependencies.
- **Local Environment Variables:** A .env file stores configuration like the database connection string, JWT secret, and admin key. (Details in Section 6.)
- **MongoDB Atlas Access:** You do not need a local MongoDB instance — the app uses MongoDB Atlas (cloud database). Have the connection URI and credentials ready.

Optional Tools:

- **Postman or HTTPie:** for testing backend API endpoints during development.
- **MongoDB Compass:** GUI for browsing the database — extremely useful for verifying entries and sharing flags.

2.4 Production Environment

For production, Mind Mirror is deployed across two cloud platforms:

- **Vercel** for the React frontend (static site).
- **Render** for the FastAPI backend (Python web service). Render is recommended over Vercel's serverless functions because the Hugging Face models need a persistent process to avoid reloading on every cold start.

You do not need to manage physical servers, but you will need to prepare the following:

- **Vercel Account:** Sign up at vercel.com (using GitHub login is convenient). Free tier is sufficient.
- **Render Account:** Sign up at render.com. The free Web Service tier works for testing; the $7/month Starter tier is recommended for production because it keeps the model warm in memory.
- **GitHub Repository Access:** Both platforms deploy directly from your GitHub repository.
- **MongoDB Atlas Cluster:** Accessible by both your local machine and Render. Whitelist 0.0.0.0/0 during initial deployment, then restrict later.
- **Optional Groq API Key:** Active account at console.groq.com if you want LLM-generated suggestions. The app gracefully falls back to rule-based suggestions if the key is omitted.

Both platforms provide default subdomains (e.g., your-app.vercel.app, your-api.onrender.com), handle SSL certificates automatically, and bind the deployments to your Git repository for continuous deployment on every push.

2.5 Optional Tools

While not strictly required, the following tools and services can assist in development and deployment:

- **Postman / HTTPie / curl** — to test and debug API endpoints by simulating requests.

- **MongoDB Atlas UI / Compass** — to view and manage the data in your database. Extremely helpful for verifying that entries have the correct sharing flags.
- **Render & Vercel Dashboards** — each provides build logs, runtime logs, and environment variable management. Familiarize yourself with both for debugging production issues.
- **Hugging Face account** — optional, but useful if you wish to swap out the default sentiment/emotion models for fine-tuned alternatives.

Having these tools can streamline troubleshooting and give more visibility into the application's behavior both locally and in production.

3. Installation and Setup (Development Environment)

This section covers the step-by-step setup for running Mind Mirror on your local machine. You will set up both the backend (FastAPI server) and the frontend (Vite + React app). By the end of this section, you should have the backend server running locally and the frontend accessible in your browser, communicating with the backend.

3.1 Setting Up the Backend (Development)

Follow these steps to set up and run the backend API locally:

1. Clone the Repository

If you haven't already, clone the project repository to your local machine using Git. In a terminal, run:

```
git clone https://github.com/<your-org>/mind-mirror.git
cd mind-mirror
```

2. Create and activate a Python virtual environment

```
cd backend
python -m venv .venv
# macOS / Linux:
source .venv/bin/activate
# Windows (PowerShell):
.venv\Scripts\Activate.ps1
```

3. Install backend dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, PyMongo (async), the Hugging Face transformers library, and all other backend dependencies. Ensure the installation completes without errors.

4. Configure backend environment variables

The backend relies on configuration values (database URI, JWT secret, admin key, optional Groq key) which should be set in a .env file in the backend/ directory. Create the file if it does not exist:

```
MONGODB_URL=mongodb+srv://<user>:<password>@<cluster>.mongodb.net
DATABASE_NAME=mindmirror
JWT_SECRET=replace-with-a-long-random-string
JWT_ALGORITHM=HS256
JWT_EXPIRATION_MINUTES=1440
ADMIN_API_KEY=pick-another-long-random-string
GROQ_API_KEY=   # leave empty to use rule-based suggestions
```

Save the .env file. Do not commit it to version control — it should be listed in .gitignore.

5. Start the backend server

```
uvicorn app.main:app --reload --port 8000
```

On first launch the server downloads the Hugging Face sentiment and emotion models (this takes 30–60 seconds and uses ~500 MB of disk). You should see `INFO: Uvicorn running on http://127.0.0.1:8000` in the console.

6. Verify the backend is running

Open `http://localhost:8000/docs` in your browser. The interactive Swagger UI should display every API endpoint. Try the `GET /` endpoint — it should return `{ "message": "MindMirror API is running" }`.

Keep the backend server running for the next steps.

3.2 Setting Up the Frontend (Development)

Now set up and run the React (Vite) frontend:

1. Install frontend dependencies

Open a new terminal window (keep the backend running in the other) and navigate to the frontend directory:

```
cd ../frontend
npm install
```

2. Configure the frontend environment

Create a .env file (or .env.local) in the frontend/ directory:

```
VITE_API_BASE_URL=http://localhost:8000/api
```

3. Start the frontend development server

```
npm run dev
```

Vite will compile the app and print the local URL — usually `http://localhost:5173`.

4. Access the frontend UI

Open the printed URL. You should see the Mind Mirror login page. Click "Sign Up" and create a test patient account, then log in. The app should redirect you to /journal.

5. Verify frontend ↔ backend integration

- Open the browser dev console (F12) and switch to the Network tab.
- Submit a journal entry. You should see a `POST /api/checkin` request returning 201 Created.
- Watch the backend terminal — it should log the request and the analysis output (sentiment + emotion labels).

If requests fail with CORS errors, ensure your backend's CORS allowed-origins list includes your Vite dev URL (defaults are http://localhost:5173 and http://127.0.0.1:5173).

3.3 Testing the Local Setup

Before moving on to deployment, test the main features locally:

- **Basic Navigation:** Click through the frontend UI to ensure all pages (Journal, Insights, Past Entries, Settings) load without errors.
- **Journal entry creation:** Submit a few entries with different moods and reflections. Check that they appear under Past Entries with correct sentiment labels.
- **Sentiment analysis:** Confirm that entries get a sentiment label (positive/neutral/negative) and a suggestion. The suggestion should be color-coded based on the entry's sentiment.
- **Per-entry sharing:** Toggle the share-with-therapist switch on an entry and verify the badge appears. In MongoDB Compass, confirm that `shared_with_therapist` was set to true on that document.
- **Settings page:** Open Settings and try entering a fake therapist ID — it should fail gracefully.
- **Logging and Errors:** Watch the terminal output for both backend and frontend. Address any errors before proceeding.

By thoroughly testing locally, you can be confident that your configuration is correct before proceeding to deploy. Once everything looks good, stop the development servers (Ctrl+C in each terminal) and prepare for deployment.

4. Database Configuration (MongoDB Atlas)

Mind Mirror uses MongoDB Atlas to store user accounts, journal entries (check-ins), patient profiles, and therapist–patient links. In development the connection URI is referenced as MONGODB_URL in your .env.

4.1 MongoDB Atlas Setup

If you do not yet have a MongoDB Atlas cluster, follow these steps:

1. Create a MongoDB Atlas account at https://cloud.mongodb.com and create a new project.
2. Click "Build a Database" and select the free M0 cluster tier. Pick a cloud provider and region close to your users.
3. Under Security → Database Access, create a new database user with read/write privileges. Note the username and password — you will need them for the connection string.
4. Under Security → Network Access, add IP address 0.0.0.0/0 for development access (this allows connections from anywhere). For production, you should restrict this to Render's outbound IP ranges, or leave it open with strong authentication.
5. Under Database → Connect → "Connect your application," copy the connection string. It will look like:

```
mongodb+srv://<username>:<password>@cluster0.abcde.mongodb.net/?
retryWrites=true&w=majority
```

6. Replace <username> and <password> with your actual values, and add this to the backend's .env as MONGODB_URL.
7. Set DATABASE_NAME in the .env to mindmirror (or whatever you prefer).

Important: If your password contains special characters (`@ : / ? #`), they must be URL-encoded.

4.2 Verifying Database Connection

With the backend running locally, you should see a startup log line confirming the connection. If you see a connection error, check that:

- The URI is correct (no typos in cluster name, user, or password).
- The database user has the correct role (readWrite on the mindmirror database, or admin).
- Your IP is allowed in Atlas Network Access.
- No local firewall is blocking outbound TCP 27017 / 27018.

You can also verify the connection by hitting `GET /api/checkins` from Swagger after creating a test account — if the call returns 200, the database is reachable.

4.3 Initial Migrations

Mind Mirror has one one-time migration that creates the patient_profiles collection seeded from existing users. After your first deployment, run the migration once:

```
cd backend
python -m migrations.2026_04_22_add_patient_profiles
```

This script creates indexes on therapist_patient_links and patient_profiles, and seeds a profile document for every existing user with sharing_enabled=false (opt-in). It is idempotent — safe to run again — and prints how many profiles were seeded.

MongoDB will create the remaining collections (users, checkins, therapist_patient_links) automatically the first time the application writes to them. No additional migrations are required.

5. Third-Party Services Configuration

Apart from the database, Mind Mirror integrates with two main services: the Hugging Face transformer models (sentiment and emotion analysis) and the Groq API (optional, for LLM-generated suggestions). This section explains how to set up each and what credentials are needed.

5.1 Hugging Face Models (Sentiment & Emotion)

The backend automatically loads two transformer models from Hugging Face on startup:

- A sentiment classifier (positive / neutral / negative).
- An emotion classifier (joy, sadness, anger, fear, etc.).

No account or API key is required for the default models — they are downloaded directly from Hugging Face's public model hub the first time the backend runs, and cached locally under ~/.cache/huggingface. Subsequent launches load from this cache and do not need internet access for the models.

Note: If you are deploying on a platform with limited disk (such as Vercel serverless, which has a small ephemeral filesystem), the model download will fail or time out. This is the primary reason the backend is deployed on Render rather than Vercel.

If you want to swap in a fine-tuned model, edit app/services/nlp.py and update the model identifier, then redeploy.

5.2 Groq API Configuration (Optional Suggestions)

Mind Mirror generates a personalized suggestion at the bottom of each entry. By default the backend uses a rule-based suggestion engine that picks supportive text based on the analyzed sentiment and mood score. If you want the suggestions to be LLM-generated (more varied, more conversational), connect a Groq API key:

8. Create an account at https://console.groq.com.
9. Navigate to API Keys and click "Create API Key." Copy the key — it begins with gsk_.
10. Set the GROQ_API_KEY in your backend .env file (and on Render in production).

The suggestion service uses the key only when present. If GROQ_API_KEY is empty or unset, the backend automatically falls back to the rule-based engine. No frontend changes are required.

Security note: The Groq API key is a server-side secret. Never expose it on the frontend or commit it to Git.

6. Environment Variables

Throughout the previous sections we identified several environment variables critical to running Mind Mirror. During local development these are stored in a .env file in the backend/ directory. In production they must be configured on Render (for the backend) and Vercel (for the frontend).

6.1 Summary of Environment Variables

The table below lists every environment variable used by Mind Mirror, where it is read, and an example value:

Variable	Description	Example
MONGODB_URL	MongoDB Atlas connection string (backend).	mongodb+srv://user:pass@cluster0.abcde.mongodb.net
DATABASE_NAME	MongoDB database name to use.	mindmirror
JWT_SECRET	Secret used to sign and verify JWT tokens. Generate with secrets.token_hex(32).	a-long-random-string
JWT_ALGORITHM	JWT signing algorithm.	HS256
JWT_EXPIRATION_MINUTES	How long access tokens remain valid.	1440
ADMIN_API_KEY	Required header X-Admin-Key value to provision therapist accounts. Disabled if blank.	another-long-random-string
GROQ_API_KEY	Optional. If set, suggestions are generated by Groq LLM. If blank, falls back to rule-based.	gsk_...
VITE_API_BASE_URL	Frontend variable. Full URL of the backend API including /api suffix.	https://mindmirror-api.onrender.com/api

All values shown above are illustrative — replace them with your actual credentials. Treat MONGODB_URL, JWT_SECRET, ADMIN_API_KEY, and GROQ_API_KEY as secrets and never commit them to source control.

6.2 Using .env in Local Development

For local development, ensure the .env file resides in the backend/ directory. The FastAPI backend uses pydantic-settings to load it automatically on startup, making variables available via the settings object.

Tips:

- **Do not commit .env to Git.** It should be listed in .gitignore. The provided project already excludes it.
- **Restart the backend** if you change .env values — pydantic-settings only loads the file on startup.
- **Generate strong secrets.** For JWT_SECRET and ADMIN_API_KEY, run `python -c "import secrets; print(secrets.token_hex(32))"` to generate a secure random value.

6.3 Configuring Environment Variables on the Cloud

When deploying, you cannot use the local .env file directly. Instead you configure these values on each platform:

On Render (Backend):

11. Open your backend service in the Render dashboard.
12. Navigate to Environment in the left sidebar.
13. Click "Add Environment Variable" and add every variable from section 6.1 EXCEPT VITE_API_BASE_URL (that's a frontend variable). Render exposes these as standard environment variables to your Python process.
14. Render automatically redeploys the service when you change environment variables.

On Vercel (Frontend):

15. Open your frontend project in the Vercel dashboard.
16. Go to Settings → Environment Variables.
17. Add VITE_API_BASE_URL with the production backend URL (e.g., https://mindmirror-api.onrender.com/api). Apply it to the Production environment (and Preview if desired).
18. Trigger a redeploy. Vite reads VITE_-prefixed variables at build time only, so the value is baked into the static bundle.

Important: Vite requires the VITE_ prefix for any variable that is exposed to the browser. Variables without this prefix will NOT be accessible from frontend code. This is intentional for security — it prevents accidental leakage of backend secrets.

7. Running the Application (Local Verification)

Before deploying to production it is important to run the entire application locally and verify that all components work together. This ensures any configuration issues can be caught early. Here is a checklist for verifying the running application in development:

7.1 Starting the Backend

In a terminal, with your virtualenv activated:

```
cd backend
uvicorn app.main:app --reload --port 8000
```

You should see startup logs ending with "Application startup complete" and "Uvicorn running on http://127.0.0.1:8000". The first launch will be slower because the Hugging Face models are downloaded and loaded.

7.2 Starting the Frontend

In a second terminal:

```
cd frontend
npm run dev
```

Vite prints the local URL (typically `http://localhost:5173`). Open it in your browser.

7.3 Integration Testing

Walk through the core flows to confirm everything is wired together:

- **Sign up:** Create a patient account. You should be redirected to /journal.
- **Submit an entry:** Pick a mood emoji, write a few sentences, click Submit. Verify the entry appears under Past Entries with a sentiment label and a suggestion.
- **Verify color-coded suggestions:** Submit one positive, one neutral, and one negative entry. The suggestion box should be green, blue, and coral respectively.
- **Per-entry sharing:** Open Past Entries → toggle the share-with-therapist switch on one entry. The badge should appear in the detail view.
- **Therapist provisioning:** Use the admin endpoint (Section 8) to create a therapist account.
- **Therapist login:** Log out, then log in with the therapist's credentials. You should be redirected to /therapist/dashboard. The patient list should be empty initially.
- **Linking:** Log in as the patient again, go to Settings, paste the therapist's ID, click "Link therapist," and enable global sharing. Then log in as the therapist again — the patient should appear in the dashboard. Click them to view their shared entries and mood trend.
- **Sharing OFF behavior:** As the patient, disable global sharing in Settings. As the therapist, click the patient — you should see the "Access restricted" card.

If all of these work locally, your configuration is correct and you can deploy with confidence.

8. Provisioning Therapist Accounts

Therapist accounts are clinically privileged — they can view the journal entries and mood trends of any patient who has linked them and enabled sharing. To prevent abuse, Mind Mirror does NOT allow self-registration of therapist accounts. Instead, therapist accounts are provisioned by an administrator using a protected admin endpoint.

8.1 Why Therapists Cannot Self-Register

The public POST /api/register endpoint hardcodes role="patient" on every account it creates. This is a deliberate security boundary: regardless of what a malicious client sends in the request body, the role can never be elevated to therapist through the public API.

Therapist accounts are created exclusively through POST /api/admin/therapists, which requires the X-Admin-Key header. The server validates this header against the ADMIN_API_KEY environment variable using a constant-time comparison. If the admin key is empty or missing on the server, the endpoint returns 503 Service Unavailable — the feature is disabled by default.

8.2 Creating a Therapist Account

As the system administrator, you create therapist accounts on behalf of clinicians. The recommended workflow:

19. Make sure ADMIN_API_KEY is set in your backend environment (.env locally, or Render env vars in production).
20. Use curl, Postman, or the Swagger UI to call the admin endpoint.

Example using curl:

```
curl -X POST https://<your-backend-domain>/api/admin/therapists \
  -H "X-Admin-Key: <your-admin-key>" \
  -H "Content-Type: application/json" \
  -d '{"name": "Dr. Smith", "email": "drsmith@clinic.com", "password":
"clinicPass1"}'
```

The response body includes the new therapist's ID:

```
{
  "id": "6789abcd1234ef5678901234",
  "name": "Dr. Smith",
  "email": "drsmith@clinic.com",
  "role": "therapist",
  "message": "Therapist account created. Share the ID with their patients."
}
```

8.3 Sharing the Therapist ID

After provisioning, share two pieces of information with the therapist directly (out-of-band — for example, in person or via secure email):

- Their login credentials (email + initial password). They should change the password on first login (TODO if not yet implemented).
- Their therapist ID — the long hexadecimal string in the id field of the response.

The therapist then shares their ID with patients who want to link their account. Patients enter the therapist's ID in the Settings page on their dashboard. Once linked, the patient must additionally toggle the global sharing switch to grant access.

Tip: After the therapist logs in for the first time, they can also retrieve their own ID by calling `GET /api/me` — the response includes the id field. The therapist dashboard also displays this ID prominently when no patients are linked yet, making it easy to copy and share.

9. Deployment to the Cloud

Mind Mirror is deployed across two cloud platforms: Vercel for the frontend (static React/Vite build) and Render for the backend (FastAPI Python service). This split is deliberate — Render's persistent web-service tier keeps the Hugging Face models loaded in memory, avoiding the cold-start penalty that would occur on a serverless platform like Vercel Functions.

9.1 Frontend Deployment (Vercel)

21. Push your code to GitHub if you have not already.
22. Log in to Vercel and click "Add New… → Project." Select your Mind Mirror repository.
23. In the project configuration, set Root Directory to frontend.
24. Vercel auto-detects Vite. The defaults are correct: Build Command npm run build, Output Directory dist, Install Command npm install.
25. Add the environment variable VITE_API_BASE_URL with the value of your Render backend URL plus /api (e.g., https://mindmirror-api.onrender.com/api). You will get the actual URL after deploying the backend in section 9.2 — set a placeholder for now and update it later.
26. Click Deploy. After ~1 minute, Vercel publishes the site at a default subdomain like mindmirror.vercel.app.

9.2 Backend Deployment (Render)

27. Log in to Render and click "New → Web Service." Connect your GitHub account if needed and select the Mind Mirror repository.
28. Configure the service:
    - **Name:** mindmirror-api (or any name — this becomes part of the URL).
    - **Region:** pick the region closest to your MongoDB Atlas cluster.
    - **Branch:** main.
    - **Root Directory:** backend.
    - **Runtime:** Python 3.
    - **Build Command:** `pip install -r requirements.txt`
    - **Start Command:** `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
    - **Instance Type:** Starter ($7/month) is recommended. The free tier sleeps after 15 minutes of inactivity, which forces the model to reload on the next request — adding ~30 s to the first call after idle.
29. Add every backend environment variable from section 6.1 to the Environment section. Crucially: MONGODB_URL, DATABASE_NAME, JWT_SECRET, JWT_ALGORITHM, JWT_EXPIRATION_MINUTES, ADMIN_API_KEY, and (optionally) GROQ_API_KEY.
30. Click "Create Web Service." Render builds the project and starts uvicorn. The first deploy takes ~5 minutes (because the Hugging Face models download on first run).

31. Once the service is live, copy the URL Render assigns (e.g., https://mindmirror-api.onrender.com). Test it by hitting /docs in your browser — the Swagger UI should render.

9.3 Connecting Frontend and Backend

With both services deployed, wire the frontend to the live backend:

32. In the Vercel project settings, update VITE_API_BASE_URL to your Render URL plus /api. For example: https://mindmirror-api.onrender.com/api.
33. Trigger a redeploy of the frontend (Vercel rebuilds because Vite bakes the value at build time).
34. On the backend, update the CORS allowed-origins list in app/main.py if needed to include your Vercel domain (e.g., https://mindmirror.vercel.app). Push the change to GitHub — Render auto-deploys.
35. Visit your Vercel domain and run through the same integration tests from section 7.3 — sign-up, journal entry, sharing, therapist linking. Everything should now work end-to-end against the cloud database.

Both deployments are now connected to GitHub — every push to the main branch will trigger automatic rebuilds on both platforms.

10. User Guide

This section explains how end users — both patients and therapists — interact with the deployed Mind Mirror application.

10.1 Patient Workflow

Sign Up

New patients click "Sign Up" on the login page and provide a name, email, and password. Their account is automatically created with the patient role; they cannot self-elevate to therapist. After signup they are redirected to the Journal page.

Daily Journal Entry

On the Journal page, the patient picks a mood (Awful → Great), writes an optional reflection, and clicks Submit. Next to the Submit button there is a sharing toggle — when turned ON, the entry will be visible to any linked therapist who has been granted access; when OFF (the default), the entry stays private.

After submission the backend analyzes the reflection and returns:

- A sentiment label (positive / neutral / negative).
- An emotion label (joy, sadness, anger, etc.).
- A color-coded suggestion (green for positive, blue for neutral, coral for negative).

Insights

The Insights page shows charts and summaries derived from the patient's check-ins: a 7-day mood average, sentiment distribution over time, and a brief weekly summary.

Past Entries

Every entry the patient has written is listed under Past Entries, grouped chronologically. Each entry card shows a sharing toggle — patients can flip an entry's sharing state at any time after creation. The detail view shows the full reflection plus its color-coded suggestion.

Settings — Therapist Sharing

On the Settings page, patients can:

- Toggle the global sharing switch (master ON/OFF). When OFF, no therapist can view any data, even entries marked as shared.
- Add a therapist by entering their therapist ID (provided by the therapist directly).
- Remove a previously linked therapist.
- See the list of currently linked therapists with the date each one was added.

For a therapist to actually see an entry, three conditions must all be true:

36. The therapist must be linked to the patient (Settings → Add a therapist).
37. The patient's global sharing toggle must be ON.

38. The specific entry must have shared_with_therapist = true (toggled either at submit time or later from Past Entries).

This three-layer model gives patients fine-grained control over what their therapist sees.

10.2 Therapist Workflow

First Login

Therapists log in using credentials provided by the administrator (see section 8). After login they are redirected to the Therapist Dashboard at /therapist/dashboard.

Dashboard

The dashboard greets the therapist by name and shows three stats:

- Total Patients — the number of patients who have linked the therapist.
- Actively Sharing — patients with the global sharing toggle ON.
- Sharing Paused — patients who linked but currently have sharing OFF.

Below the stats is a searchable grid of patient cards. Each card shows the patient's name, email, and a colored badge indicating whether they are currently sharing data.

If no patients are linked yet, the dashboard displays the therapist's own ID in a copyable code chip — they can share this with patients who want to link.

Patient Detail View

Clicking a patient card navigates to /therapist/patient/<patient-id>. The page has two tabs:

- **Journal Entries:** Lists every shared entry in chronological order. Each entry shows the date, time, sentiment pill, and full reflection text. Entries are read-only.
- **Mood Trend:** Three summary cards (positive/neutral/negative counts), an SVG line chart showing sentiment over time, and a detailed timeline list with confidence percentages. All read-only.

If the patient has paused sharing, the page shows an "Access Restricted" card with a friendly message — therapists never see misleading empty states or stale data.

Read-Only Access

Therapists cannot edit, delete, or comment on patient entries. The header explicitly displays a "Read-only access" badge next to each patient's record. This is enforced both in the UI (no edit/delete buttons exist) and on the backend (therapists are blocked from any mutation endpoints by the require_patient guard on those routes).

11. Troubleshooting

Despite careful setup, you may encounter issues. Below are common problems and their solutions, categorized by area.

11.1 Backend Issues

Backend won't start locally

- Verify your virtualenv is activated. The shell prompt should show (.venv).
- Re-run pip install -r requirements.txt — missing dependencies are the most common cause.
- Check Python version: Mind Mirror requires Python 3.11 or newer.
- Ensure no other process is bound to port 8000. Use lsof -i :8000 (macOS/Linux) or netstat -ano | findstr 8000 (Windows).

Cannot connect to MongoDB

- Double-check the MONGODB_URL — special characters in the password must be URL-encoded.
- Confirm your IP is allowed in Atlas Network Access. For testing, set 0.0.0.0/0.
- Make sure the cluster is running — free-tier clusters pause after long inactivity.
- If running on Render, ensure 0.0.0.0/0 is allowed (Render's outbound IPs are dynamic).

NLP models fail to load

- First-time download requires internet connectivity. Check that the server has outbound HTTPS access to huggingface.co.
- If the disk fills up, the cache (~/.cache/huggingface) may be incomplete — delete it and restart.
- On Render's free tier, the model may time out during first download. Upgrade to Starter or trigger a fresh deploy after the first attempt.

Therapist provisioning returns 503

This means ADMIN_API_KEY is unset or empty. Set it in your environment and restart the backend.

Therapist provisioning returns 401

The X-Admin-Key header you sent does not match the configured value. Verify both sides character-for-character (no trailing whitespace).

11.2 Frontend Issues

Vite won't start

- Re-run npm install — package-lock.json drift is a common cause.
- Check Node version is 18+.
- Delete node_modules and the lock file, then npm install again if errors persist.

Login redirects to /journal even for therapists

This was a known bug — the login flow now reads user.role from the response and redirects therapists to /therapist/dashboard. If you still see incorrect redirection, log out, clear browser localStorage, then log in again.

Patient cards show but no entries inside

Three things must all be true for the therapist to see entries: link is active, global sharing is ON, AND the specific entry has shared_with_therapist=true. Verify each condition. The most common cause is that the patient created entries before turning on the toggle — by default, new entries are NOT shared.

CORS errors in production

The backend's CORS allowed-origins list must include your Vercel domain. Edit app/main.py, add the domain (e.g., https://mindmirror.vercel.app), commit, and push — Render redeploys automatically.

11.3 Deployment Issues

Render build fails

- Confirm the Root Directory is set to backend, not the repository root.
- Verify the start command is exactly uvicorn app.main:app --host 0.0.0.0 --port $PORT (Render injects PORT).
- Inspect the build logs — most failures are dependency-resolution errors, fixed by pinning versions in requirements.txt.

Vercel build fails with "VITE_API_BASE_URL is undefined"

You forgot to add the variable in Settings → Environment Variables, or you added it to the wrong environment (e.g., only Preview, not Production). Add it to Production and redeploy.

Therapist dashboard shows blank after deploy

- Open the browser console and Network tab — look for failing requests.
- **Most likely cause:** VITE_API_BASE_URL still points to localhost:8000. Update it to the Render URL and redeploy.
- If requests reach the backend but return 401, the JWT token may have been signed with a different JWT_SECRET than the one currently configured. Log out and log back in to mint a new token.

Application errors only in production

- Filesystems on Linux (Render) are case-sensitive. Imports like ./Components/Foo work on Windows/macOS but fail on Linux if the actual file is ./components/Foo. Standardize file casing.
- Render's Python version may differ from your local — pin it via runtime.txt (e.g., python-3.11.7) to avoid surprises.

Logs and Monitoring

Both Render and Vercel provide live log streams in their dashboards. For the backend, every API request is logged with status code and duration. For the frontend, console errors will not appear in Vercel logs (they are client-side) — use the browser dev tools.

12. Contact Information

For further assistance, please reach out:

- Support Email: support@mindmirror.app
- GitHub Repository: https://github.com/<your-org>/mind-mirror
- Documentation: https://docs.mindmirror.app

We're here to help! For immediate support, raise an issue on our GitHub page or contact us via email. Bug reports should include the browser version, the steps to reproduce, and any relevant error messages from the browser console or Render logs.

Thank you for using Mind Mirror.